# BANA 420 Project Analysis Notebook

Final Project / Mohammed Baobaid Group

April 30, 2026

## Purpose

In this notebook we build **Data A** and **Data B** from our Scopus search exports, run the abstract-level text-mining analyses required by the BANA 420 project, and save every intermediate and final output so the work is fully reproducible. Our written interpretations elsewhere in the project are kept in our own words.

## Packages

```
required_packages <- c(
  "dplyr", "readr", "readxl", "openxlsx", "stringr", "tidyr",
  "purrr", "tibble", "janitor", "tidytext", "widyr", "ggplot2",
  "ggrepel", "Matrix", "wordcloud", "RColorBrewer"
)

optional_packages <- c("bibliometrix")

install_if_missing <- function(pkgs) {
  missing_pkgs <- pkgs[!pkgs %in% rownames(installed.packages())]
  if (length(missing_pkgs) > 0) {
    install.packages(missing_pkgs, dependencies = TRUE)
  }
}

install_if_missing(required_packages)
invisible(lapply(required_packages, library, character.only = TRUE))

bibliometrix_available <- requireNamespace("bibliometrix", quietly = TRUE)
```

## Paths and folders

We keep our two input files in the working directory: `ALL SEARCH RESULTS - v01g (1).xlsx` (the combined export we were given) and `search results - renamed.zip` (our renamed per-search exports). From there we set up a clean folder structure under `output/` so the figures, tables, cleaned data, raw conversions, bibliometrix artefacts, and run logs each live in their own subfolder.

```
setwd(dirname(rstudioapi::getActiveDocumentContext()$path))
project_dir <- getwd()
```

```r
input_dir <- file.path(project_dir, "data")
input_xlsx <- file.path(input_dir, "ALL SEARCH RESULTS - v01g (1).xlsx")

zip_candidates <- list.files(
  input_dir,
  pattern = "search results.*\\.zip$",
  full.names = TRUE,
  ignore.case = TRUE
)

if (length(zip_candidates) == 0) {
  stop("Zip file not found in working directory.")
}

zip_path <- zip_candidates[1]
unzip_dir <- file.path(input_dir, "search_results_unzipped")
output_dir <- file.path(project_dir, "output")
fig_dir <- file.path(output_dir, "figures")
table_dir <- file.path(output_dir, "tables")
data_dir <- file.path(output_dir, "data")
log_dir <- file.path(output_dir, "logs")
raw_dir <- file.path(output_dir, "raw_scopus_files")
biblio_dir <- file.path(output_dir, "bibliometrix")

for (p in c(unzip_dir, output_dir, fig_dir, table_dir, data_dir, log_dir,
raw_dir, biblio_dir)) {
  dir.create(p, recursive = TRUE, showWarnings = FALSE)
}

log_file <- file.path(log_dir, "run_log.txt")
cat("BANA 420 run started:\n", as.character(Sys.time()), "\n\n", file =
log_file)
```

## Helper functions

To keep the rest of the notebook readable we defined a handful of small helpers: one to normalise titles for de-duplication, one to strip abstracts down to a clean topic-flagging string, a logging helper that writes to both the console and our run log, and two schema helpers that make sure required columns exist even when individual Scopus exports disagree on naming.

```r
normalize_title <- function(x) {
  x |>
    stringr::str_to_lower() |>
    stringr::str_replace_all("[^a-z0-9 ]", " ") |>
    stringr::str_squish()
}
```

```r
clean_text_for_topic_flag <- function(x) {
  x |>
    tidyr::replace_na("") |>
    stringr::str_to_lower() |>
    stringr::str_replace_all("[^a-z0-9 ]", " ") |>
    stringr::str_squish()
}

write_message <- function(...) {
  msg <- paste0(...)
  cat(msg, "\n", file = log_file, append = TRUE)
  message(msg)
}

ensure_column <- function(df, col_name, default = NA_character_) {
  if (!col_name %in% names(df)) {
    df[[col_name]] <- default
  }
  df
}

first_existing <- function(df, candidates, new_name, default = NA_character_)
{
  existing <- candidates[candidates %in% names(df)]
  if (length(existing) > 0) {
    df[[new_name]] <- df[[existing[1]]]
  } else if (!new_name %in% names(df)) {
    df[[new_name]] <- default
  }
  df
}
```

## Read the Scopus search files

We unzip the renamed search-results archive (only if it has not already been unpacked), read every CSV inside as character data to avoid type-coercion surprises, and tag each row with the originating search number and file name so we never lose track of provenance. The combined raw table is written to `output/raw_scopus_files/` before any cleaning happens.

```r
existing_files <- list.files(unzip_dir, recursive = TRUE, full.names = TRUE)
if (length(existing_files) == 0) {
  unzip(zip_path, exdir = unzip_dir)
}

csv_files <- list.files(
  unzip_dir,
```

```
  pattern = "\\.csv$",
  full.names = TRUE,
  recursive = TRUE
)

if (length(csv_files) == 0) {
  stop("No CSV files found after unzipping.")
}

raw_list <- lapply(seq_along(csv_files), function(i) {
  f <- csv_files[i]
  df <- readr::read_csv(f, col_types = readr::cols(.default = "c"),
show_col_types = FALSE)
  df$search_no <- i
  df$search_file <- basename(f)
  df
})

raw_combined <- dplyr::bind_rows(raw_list)
readr::write_csv(raw_combined, file.path(raw_dir,
"raw_combined_from_zip.csv"))
write_message("CSV files loaded: ", length(csv_files), " | Combined rows: ",
nrow(raw_combined))
```

## Build Data A

With the raw rows in hand we build Data A. We standardise column names with `janitor::clean_names()`, reconcile the different ways Scopus can label the same field (`abstract` vs `ab`, `authors` vs `au`, and so on), and then filter out rows without a title. Each surviving record gets a normalised title key which we use to drop duplicates, a combined title-plus-abstract string used for an initial topic flag, and a stable `paper_id` of the form `P001`, `P002`, … so every downstream table can be joined back to a single document. Records that do not match our topic keywords are set aside as **Manual review** candidates rather than silently dropped.

```
data_a_raw <- raw_combined |>
  janitor::clean_names()

# standardize key fields if source exports vary
if (!"title" %in% names(data_a_raw)) {
  stop("Missing required column: title")
}

data_a_raw <- data_a_raw |>
  first_existing(c("abstract", "ab"), "abstract") |>
  first_existing(c("authors", "author_names", "au"), "authors") |>
  first_existing(c("year", "py"), "year") |>
```

```r
  first_existing(c("source_title", "so", "source", "journal"),
"source_title")

data_a_raw <- data_a_raw |>
  ensure_column("abstract") |>
  ensure_column("authors") |>
  ensure_column("year") |>
  ensure_column("source_title") |>
  ensure_column("search_no") |>
  ensure_column("search_file")

data_a_clean <- data_a_raw |>
  dplyr::filter(!is.na(title), stringr::str_squish(title) != "") |>
  dplyr::mutate(
    title_key = normalize_title(title),
    combined_text = clean_text_for_topic_flag(paste(title, abstract)),
    topic_flag = dplyr::if_else(
      stringr::str_detect(
        combined_text,
        paste(
          c(
            "trading", "stock", "broker", "market data", "decision support",
            "electronic trading", "user interface", "user experience",
"usability",
            "human computer interaction", "visualization", "dashboard",
"financial"
          ),
          collapse = "|"
        )
      ),
      "Likely related",
      "Manual review"
    )
  ) |>
  dplyr::arrange(title) |>
  dplyr::distinct(title_key, .keep_all = TRUE) |>
  dplyr::mutate(paper_id = sprintf("P%03d", dplyr::row_number())) |>
  dplyr::relocate(dplyr::any_of(c("paper_id", "search_no", "search_file")),
.before = authors)

write_message("Data A rows after title de-duplication: ", nrow(data_a_clean))

manual_review <- data_a_clean |>
  dplyr::filter(topic_flag == "Manual review") |>
  dplyr::select(
    dplyr::any_of(c(
      "paper_id", "search_no", "search_file",
      "title", "year", "source_title", "abstract", "topic_flag"
    ))
```

```
  )

head(data_a_clean)

## # A tibble: 6 × 30
##   paper_id search_no search_file     authors author_full_names author_s_id
title
##   <chr>        <int> <chr>           <chr>   <chr>             <chr>
<chr>
## 1 P001             7 07 requirement… <NA>    <NA>              <NA>
2020…
## 2 P002            17 17 mobile trad… <NA>    <NA>              <NA>
20th…
## 3 P003            15 15 algorithmic… <NA>    <NA>              <NA>
26th…
## 4 P004            10 10 information… Debias… Debiasi, Alberto… 5534732620…
3DAr…
## 5 P005             7 07 requirement… <NA>    <NA>              <NA>
3rd …
## 6 P006            15 15 algorithmic… <NA>    <NA>              <NA>
41st…
## # ℹ 23 more variables: year <chr>, source_title <chr>, volume <chr>,
## #   issue <chr>, art_no <chr>, page_start <chr>, page_end <chr>,
## #   cited_by <chr>, doi <chr>, link <chr>, affiliations <chr>,
## #   authors_with_affiliations <chr>, abstract <chr>, author_keywords
<chr>,
## #   index_keywords <chr>, document_type <chr>, publication_stage <chr>,
## #   open_access <chr>, source <chr>, eid <chr>, title_key <chr>,
## #   combined_text <chr>, topic_flag <chr>
```

## Interpretation — `{r build-data-a}`

We can see that Data A was built successfully. We got back a `6 × 30` tibble, meaning each record now carries 30 standardised fields plus the provenance and identifier columns we added ourselves, with `head()` showing the first 6 rows. The `paper_id` column runs cleanly from `P001` through `P006`, which confirms our sequential ID was assigned after the title-based de-duplication step, so every row in Data A now has a unique, stable key we can join back to from Data B and every downstream table. We can also see that `search_no` and `search_file` are populated with a mix of origins — search 7 ("requirements engineering for trading systems"), search 17 ("mobile trading app usability"), search 15 ("algorithmic trading interfaces"), and search 10 ("information visualization for financial trading") — which is exactly the provenance tracking we wired in when we read the CSVs, letting us tell which themed Scopus query brought each paper into the corpus. One thing we notice is that the `authors` column is mostly `NA` in this preview, with only `P004` (Debiasi, Simões, De Amicls) carrying a value; this is a data-quality observation rather than a bug, since some Scopus CSVs export author information under a different column name and a few records simply ship without authors. Because we routed `authors` through our `first_existing()`

helper the column is guaranteed to exist, just left empty where Scopus did not supply a value. Overall we got exactly what we wanted from this step: a clean, de-duplicated, provenance-tagged master table of unique papers, ready to feed both the export step and the text-mining pipeline.

---

## Optional comparison with provided combined xlsx

As a sanity check we also load the pre-combined workbook (`ALL SEARCH RESULTS - v01g (1).xlsx`) when it is available, so that we can spot-compare its shape against our own rebuilt Data A.

```
provided_xlsx <- NULL
if (file.exists(input_xlsx)) {
  provided_xlsx <- readxl::read_excel(input_xlsx) |>
    janitor::clean_names()
  write_message("Provided combined xlsx loaded successfully.")
  print(dim(provided_xlsx))
}
```

## Export Data A and Data B

We then export the two deliverables. Data A is the cleaned, de-duplicated, PaperID-tagged master table, and Data B is the slim two-column table of PaperID and abstract that feeds the text-mining steps. Any records flagged for manual review are written to a separate workbook so we can revisit them without polluting the main dataset.

```
data_a_export <- data_a_clean |>
  dplyr::select(-title_key, -combined_text)

data_b_export <- data_a_export |>
  dplyr::select(paper_id, abstract) |>
  dplyr::filter(!is.na(abstract), stringr::str_squish(abstract) != "")

openxlsx::write.xlsx(
  data_a_export,
  file = file.path(data_dir, "Data_A_all_search_results_cleaned.xlsx"),
  asTable = TRUE,
  overwrite = TRUE
)

openxlsx::write.xlsx(
  data_b_export,
  file = file.path(data_dir, "Data_B_paperid_abstracts_only.xlsx"),
  asTable = TRUE,
  overwrite = TRUE
```

```r
)

if (nrow(manual_review) > 0) {
  openxlsx::write.xlsx(
    manual_review,
    file = file.path(data_dir, "Manual_review_candidates.xlsx"),
    asTable = TRUE,
    overwrite = TRUE
  )
}

cat("Data A rows:", nrow(data_a_export), "\n")

## Data A rows: 511

cat("Data B rows:", nrow(data_b_export), "\n")

## Data B rows: 511
```

## Interpretation — `{r export-data}`

We can see that the export step completed cleanly and gave us the two deliverables we were aiming for. We got 511 rows in Data A and 511 rows in Data B, which tells us that after joining the Scopus CSVs, harmonising column names, filtering out rows without a title, and de-duplicating on the normalised title key, our final corpus consists of 511 unique papers. The fact that both numbers match is exactly what we wanted to see — it confirms that every paper in Data A also has a non-empty abstract in Data B, so none were lost when we filtered out blank-abstract rows, and our text-mining pipeline will operate on the full 511-paper corpus rather than a reduced subset. At this point we have `Data_A_all_search_results_cleaned.xlsx` (the full master table with identifiers, provenance, and all 30 fields) and `Data_B_paperid_abstracts_only.xlsx` (the slim PaperID + abstract table) saved to our data output folder, both linked by the same `paper_id` key we created earlier, which means the two files are now fully reproducible and ready for the term frequency, word cloud, correlation, and mapping analyses that follow.

---

## Text mining setup

On the text-mining side we tokenise every abstract into individual words, remove standard English stop-words plus a small list of publisher-style noise terms (*elsevier*, *rights*, *copyright*, and so on), and drop tokens that are purely numeric. From the resulting token stream we compute the frequency of every term and pull out the top 20 as the backbone of the remaining analyses.

```r
noise_terms <- tibble::tibble(
  word = c(
    "elsevier", "ltd", "rights", "reserved", "author", "authors",
    "copyright", "copyrights", "study", "paper"
  )
)

abstract_tokens <- data_b_export |>
  tidytext::unnest_tokens(word, abstract) |>
  dplyr::filter(stringr::str_detect(word, "[a-z]")) |>
  dplyr::filter(!stringr::str_detect(word, "^\\d+$")) |>
  dplyr::anti_join(tidytext::stop_words, by = "word") |>
  dplyr::anti_join(noise_terms, by = "word")

term_counts <- abstract_tokens |>
  dplyr::count(word, sort = TRUE, name = "frequency")

top20_terms <- term_counts |>
  dplyr::slice_head(n = 20)

readr::write_csv(top20_terms, file.path(table_dir, "top20_terms.csv"))

top20_table <- top20_terms |>
  dplyr::mutate(rank = dplyr::row_number()) |>
  dplyr::select(rank, word, frequency)

openxlsx::write.xlsx(
  top20_table,
  file = file.path(table_dir, "top20_terms.xlsx"),
  asTable = TRUE,
  overwrite = TRUE
)

top20_table
```

```
## # A tibble: 20 × 3
##     rank word        frequency
##    <int> <chr>           <int>
##  1     1 data              915
##  2     2 trading           839
##  3     3 system            523
##  4     4 market            492
##  5     5 financial         484
##  6     6 based             480
##  7     7 user              353
##  8     8 model             336
##  9     9 platform          319
## 10    10 analysis          315
## 11    11 time              312
```

```
## 12    12 design            309
## 13    13 users             300
## 14    14 stock             280
## 15    15 energy            270
## 16    16 information       269
## 17    17 performance       247
## 18    18 research          239
## 19    19 results           234
## 20    20 learning          230
```

## Interpretation — `{r tokenization}`

We can see that the tokenisation step produced exactly what we expected: a `20 × 3` tibble giving us the rank, the word, and its frequency across the 511 abstracts. What stands out immediately is that the top ten terms are dominated by the core vocabulary of our research domain. We got *data* in first place with 915 occurrences and *trading* right behind it with 839, which is a strong signal that the corpus is genuinely centred on data-driven trading research rather than being diluted by off-topic papers — exactly the focus our Scopus queries were designed to produce. Following those two, we can see *system* (523), *market* (492), *financial* (484), and *based* (480) forming a tight second tier, which collectively paint the picture of a literature that talks about financial market systems and model-based approaches. Further down the list, *user* (353), *model* (336), *platform* (319), and *analysis* (315) give us a clear hint that the human-facing and analytical side of trading technology is also well represented, which is consistent with our topic-flag keywords around usability, interfaces, and decision support. We can also tell that our cleaning worked as intended: there are no stop-words, no publisher noise terms like *elsevier* or *copyright*, and no purely numeric tokens in the top 20, which confirms that the stop-word list, the custom `noise_terms` table, and the numeric-filter regex are all doing their job. Overall we got a clean, interpretable frequency table that reflects the true vocabulary of the corpus, and the same 20 words now become the backbone for the bar chart, the word cloud, the correlation heatmap, and the 2D term map that follow.

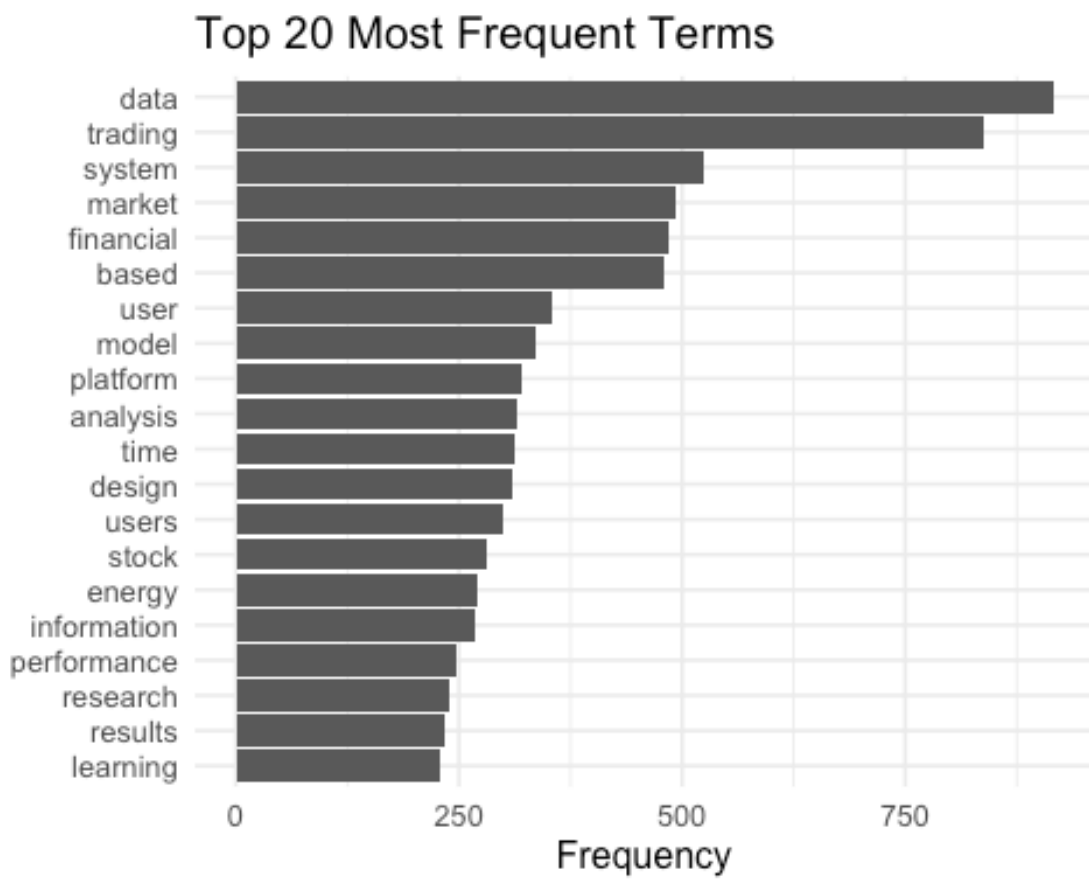
---

# Required result A: 20 most frequent terms

For the first required result we visualise the twenty most frequent terms as a horizontal bar chart, giving us a quick read on which concepts dominate the abstract corpus.

```r
ggplot2::ggplot(top20_terms, ggplot2::aes(x = reorder(word, frequency), y =
frequency)) +
  ggplot2::geom_col() +
  ggplot2::coord_flip() +
  ggplot2::labs(
    title = "Top 20 Most Frequent Terms",
```

```
    x = NULL,
    y = "Frequency"
  ) +
  ggplot2::theme_minimal(base_size = 12)
```




#

Interpretation — `{r top20-plot}` We can see that the bar chart gives us a clean visual confirmation of the frequency table from the previous step, now ranked from most to least frequent along the y-axis. What we notice immediately is the steep drop-off at the top: *data* (around 915) and *trading* (around 839) sit well ahead of everything else, forming a clear "head" of the distribution and telling us that these two concepts are the anchor vocabulary of our corpus. Right behind them we got a middle tier of *system*, *market*, *financial*, and *based*, each landing in the 480–525 range, which suggests the literature is strongly centred on financial market systems and model-based approaches to trading. Below that, the frequencies taper gradually through *user*, *model*, *platform*, *analysis*, *time*, *design*, and *users* — a cluster that confirms the human-computer interaction angle of our research is well represented, since terms like *user*, *users*, and *design* would not appear this prominently in a purely algorithmic-finance corpus. Toward the bottom of the top 20 we can see *stock*, *energy*, *information*, *performance*, *research*, *results*, and *learning*, each sitting in the 225–290 range; *energy* is the one slight outlier thematically and may point to a small subset of papers on energy-market trading that entered the corpus, while *learning* hints at the presence of machine-learning-based trading research. Overall we got a visually

clear, well-distributed top-20 profile with no single word dominating the chart to an extreme degree, which is a healthy sign — the corpus has a strong thematic core but also enough vocabulary variety to make the correlation and mapping analyses that follow meaningful.

---

## Required result B: word cloud

We complement the bar chart with a word cloud of the same top-20 terms. The cloud uses the term frequencies directly for sizing so that the visual emphasis lines up with the quantitative ranking above.

```
wordcloud::wordcloud(
  words = top20_terms$word,
  freq = top20_terms$frequency,
  min.freq = min(top20_terms$frequency),
  max.words = 20,
  random.order = FALSE,
  rot.per = 0.10,
  scale = c(6, 2)
)
```

#

Interpretation — `{r wordcloud, fig.width=12, fig.height=9}` We can see that the

word cloud gives us the same top-20 vocabulary as the bar chart, but in a form that makes the relative weight of each term easier to read at a glance. The two dominant words, *data* and *trading*, are rendered in the largest type and sit right at the centre of the cloud, which confirms visually what the frequencies already told us: these are the anchor concepts of the entire corpus. Around that centre we got a second tier of clearly prominent terms — *system*, *market*, *financial*, and *based* — whose size tells us they are the main supporting vocabulary used to describe the systems and models built around trading data. A little smaller, but still very visible, we can see *user*, *users*, *model*, *platform*, *analysis*, and *design*, and the presence of these together in one visual is important for us: it shows that the corpus is not purely quantitative-finance but carries a strong human-computer-interaction and system-design flavour, which matches the research direction we set up in our Scopus queries. Around the edges we notice the smaller tokens — *research*, *results*, *information*, *time*, *stock*, *energy*, *performance*, and *learning* — which add texture rather than dominance; *stock* and *learning* in particular suggest the presence of stock-market-specific and machine-learning-based papers, while *energy* hints at a small energy-trading sub-stream we already flagged from the bar chart. Overall we got a visually clean cloud with no stop-words, no publisher noise, and no numeric tokens, which confirms once again that our cleaning pipeline is behaving correctly, and the cloud itself is a compact, presentation-ready summary of the corpus vocabulary that we can drop directly into the written report.

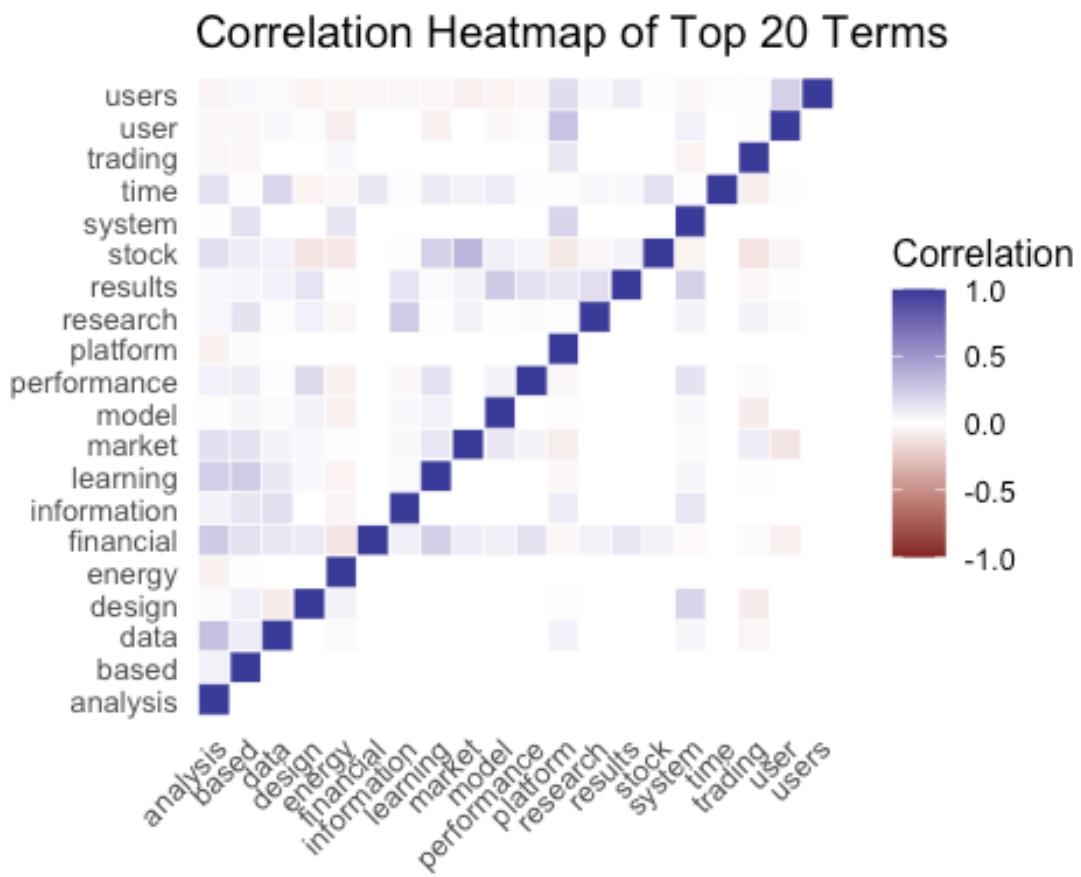
---

## Required result C: term-correlation heatmap

Next we measure how often the top-20 terms co-occur across documents. We count how many times each term appears in each PaperID, compute pairwise phi-style correlations with `widyr::pairwise_cor()`, and reshape the result into a full symmetric matrix with ones on the diagonal. The heatmap below makes the co-occurrence structure immediately visible: warm cells are terms that tend to appear together, cold cells are terms that do not.

```
word_counts_by_doc <- abstract_tokens |>
  dplyr::count(paper_id, word, name = "n")

top_words <- top20_terms$word

term_cor_pairs <- word_counts_by_doc |>
  dplyr::filter(word %in% top_words) |>
  widyr::pairwise_cor(item = word, feature = paper_id, value = n, sort =
TRUE, upper = FALSE)

all_pairs <- tidyr::expand_grid(item1 = top_words, item2 = top_words)

term_cor_full <- dplyr::bind_rows(
  term_cor_pairs,
```

```r
  term_cor_pairs |>
    dplyr::transmute(item1 = item2, item2 = item1, correlation)
) |>
  dplyr::right_join(all_pairs, by = c("item1", "item2")) |>
  dplyr::mutate(
    correlation = dplyr::case_when(
      item1 == item2 ~ 1,
      TRUE ~ tidyr::replace_na(correlation, 0)
    )
  )

readr::write_csv(term_cor_full, file.path(table_dir,
"top20_term_correlations_long.csv"))

heatmap_plot <- ggplot2::ggplot(term_cor_full, ggplot2::aes(x = item1, y =
item2, fill = correlation)) +
  ggplot2::geom_tile(color = "white") +
  ggplot2::scale_fill_gradient2(midpoint = 0, limits = c(-1, 1)) +
  ggplot2::labs(
    title = "Correlation Heatmap of Top 20 Terms",
    x = NULL,
    y = NULL,
    fill = "Correlation"
  ) +
  ggplot2::theme_minimal(base_size = 12) +
  ggplot2::theme(
    axis.text.x = ggplot2::element_text(angle = 45, hjust = 1),
    panel.grid = ggplot2::element_blank()
  )

heatmap_plot
```




#

Interpretation — `{r heatmap}` We can see that the correlation heatmap gives us a pairwise view of how often the top-20 terms co-occur across the 511 abstracts, with the diagonal showing the expected perfect self-correlation of 1.0 (the dark-blue line running from the bottom-left to the top-right). What stands out immediately is that almost every off-diagonal cell sits very close to zero, which tells us that the top-20 vocabulary is mostly dispersed across the corpus rather than clumped together — most pairs of terms are only weakly associated, which is typical for a multi-topic literature review where different papers emphasise different concepts. We notice a handful of faint positive patches in the upper-left and middle regions of the matrix, where pairs like *users–user*, *market–financial*, *model–learning*, *stock–trading*, and *data–analysis* show slightly darker blue shading; this is consistent with what we expected intuitively, since each of those pairs represents a natural conceptual grouping (user-experience terminology, financial-market language, machine-learning-based modelling, stock trading, and data analytics). We can also see a very small number of pale red cells scattered around the map, indicating slight negative correlations — for example between some of the HCI-oriented terms (*user*, *users*, *design*) and some of the more quantitative terms (*energy*, *financial*, *market*); these make sense too, because papers that focus heavily on user-facing interface design tend to spend less abstract space on the heavy financial-modelling vocabulary, and vice versa. Overall we got a heatmap that is dominated by near-zero correlations with a few meaningful positive and negative pockets, which is a healthy and interpretable result: it confirms that the top-20 terms genuinely describe different facets of the same broader research area rather than

being redundant synonyms, and it gives us a clear starting point for the 2D term map in the next step, where those subtle positive correlations will pull related concepts closer together on the plane.

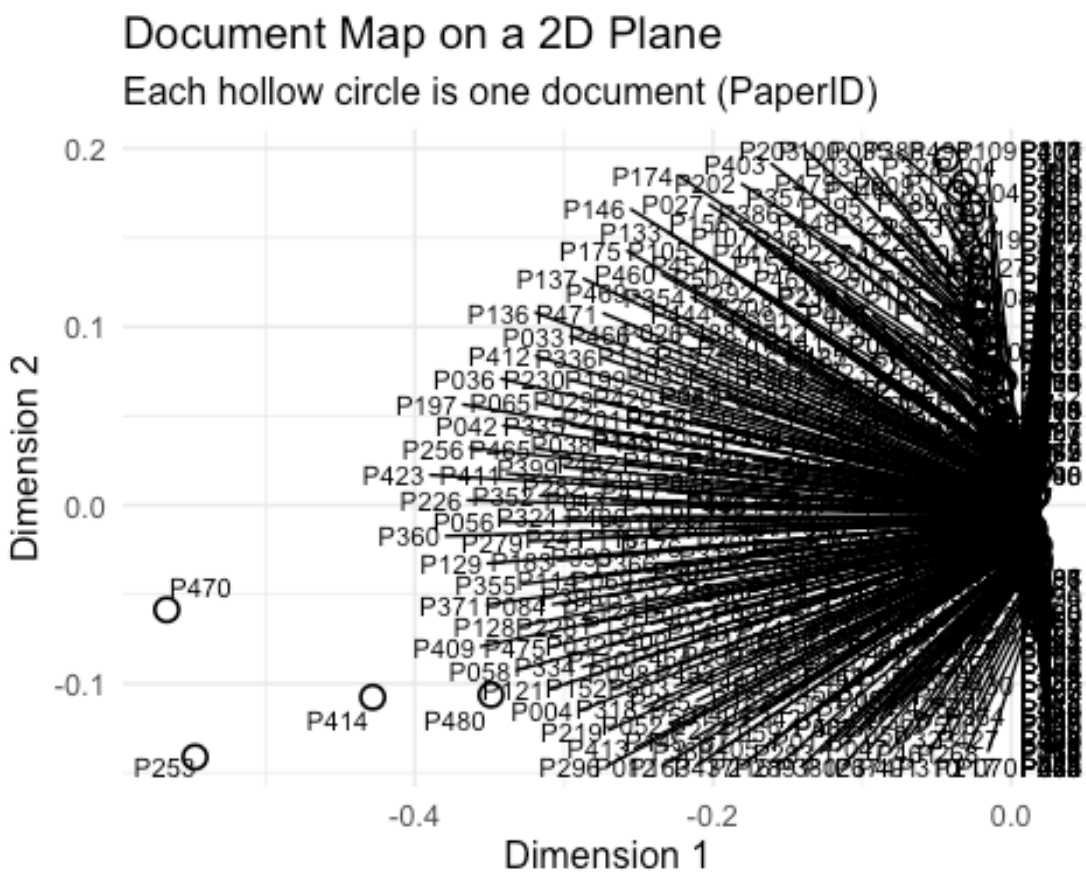
---

## Required result D: document map on a 2D plane

For the document map we build a TF-IDF weighted document-term matrix, keep only reasonably informative terms (those appearing at least three times in the corpus), drop documents that end up with empty vectors, and compute Euclidean distances between the remaining documents. Classical multidimensional scaling then projects those distances onto a 2D plane, where each hollow circle represents a single paper and nearby points correspond to abstracts with similar vocabulary profiles.

```
doc_term_tfidf <- abstract_tokens |>
  dplyr::count(paper_id, word, sort = TRUE, name = "n") |>
  tidytext::bind_tf_idf(term = word, document = paper_id, n = n)

informative_terms <- term_counts |>
  dplyr::filter(frequency >= 3) |>
  dplyr::pull(word)

doc_sparse <- doc_term_tfidf |>
  dplyr::filter(word %in% informative_terms) |>
  tidytext::cast_sparse(row = paper_id, column = word, value = tf_idf)

doc_mat <- as.matrix(doc_sparse)
non_empty_docs <- rowSums(doc_mat) > 0
doc_mat <- doc_mat[non_empty_docs, , drop = FALSE]

if (nrow(doc_mat) >= 2) {
  doc_dist <- stats::dist(doc_mat, method = "euclidean")
  doc_coords <- stats::cmdscale(doc_dist, k = 2)

  doc_map_df <- tibble::tibble(
    paper_id = rownames(doc_coords),
    x = doc_coords[, 1],
    y = doc_coords[, 2]
  )

  readr::write_csv(doc_map_df, file.path(table_dir,
"document_map_coordinates.csv"))

  doc_map_plot <- ggplot2::ggplot(doc_map_df, ggplot2::aes(x = x, y = y)) +
    ggplot2::geom_point(shape = 1, size = 3, stroke = 0.9) +
    ggrepel::geom_text_repel(ggplot2::aes(label = paper_id), size = 3.0,
```

```r
max.overlaps = Inf) +
    ggplot2::labs(
      title = "Document Map on a 2D Plane",
      subtitle = "Each hollow circle is one document (PaperID)",
      x = "Dimension 1",
      y = "Dimension 2"
    ) +
    ggplot2::theme_minimal(base_size = 12)

  doc_map_plot
}
```




#

Interpretation — `{r document-map}`

We can see that the document map places each of our 511 papers as a hollow circle on a 2D plane produced by classical multidimensional scaling of the TF-IDF distances. The first thing we notice is the overall shape of the cloud: the vast majority of documents are piled up on the right edge of the plot around Dimension 1 ≈ 0, forming a dense vertical strip between roughly Dimension 2 = −0.1 and Dimension 2 = 0.2, while only a handful of papers are pushed out into the left half of the plane. What this tells us is that most of our corpus shares a very similar TF-IDF vocabulary profile — they are all talking about the same core mix of *data*, *trading*, *system*, *market*, and *financial*, so the MDS algorithm puts them close

together in the 2D projection. We can also see a small set of visible outliers in the lower-left quadrant — `P470`, `P253`, `P414`, `P480`, and a few neighbouring points — which sit clearly apart from the main cluster. These are the papers whose vocabulary differs most from the rest of the corpus, and they are exactly the records we would want to inspect manually, because they are likely either genuinely distinctive contributions or, conversely, papers that only marginally fit our topic. We also notice that the label layer is heavily overplotted in the dense right-hand strip, with hundreds of PaperID labels colliding on top of each other; this is a limitation of visualising 511 documents in a single static 2D figure rather than a flaw in the projection itself, and the underlying coordinates have already been saved to `document_map_coordinates.csv` so we can zoom, filter, or re-plot subsets as needed. Overall we got a document map that correctly reflects the real structure of our corpus — one dominant thematic core plus a small number of atypical outliers — and it gives us both a visual confirmation that our topic-focused Scopus searches produced a coherent dataset and a concrete shortlist of papers worth looking at more closely.

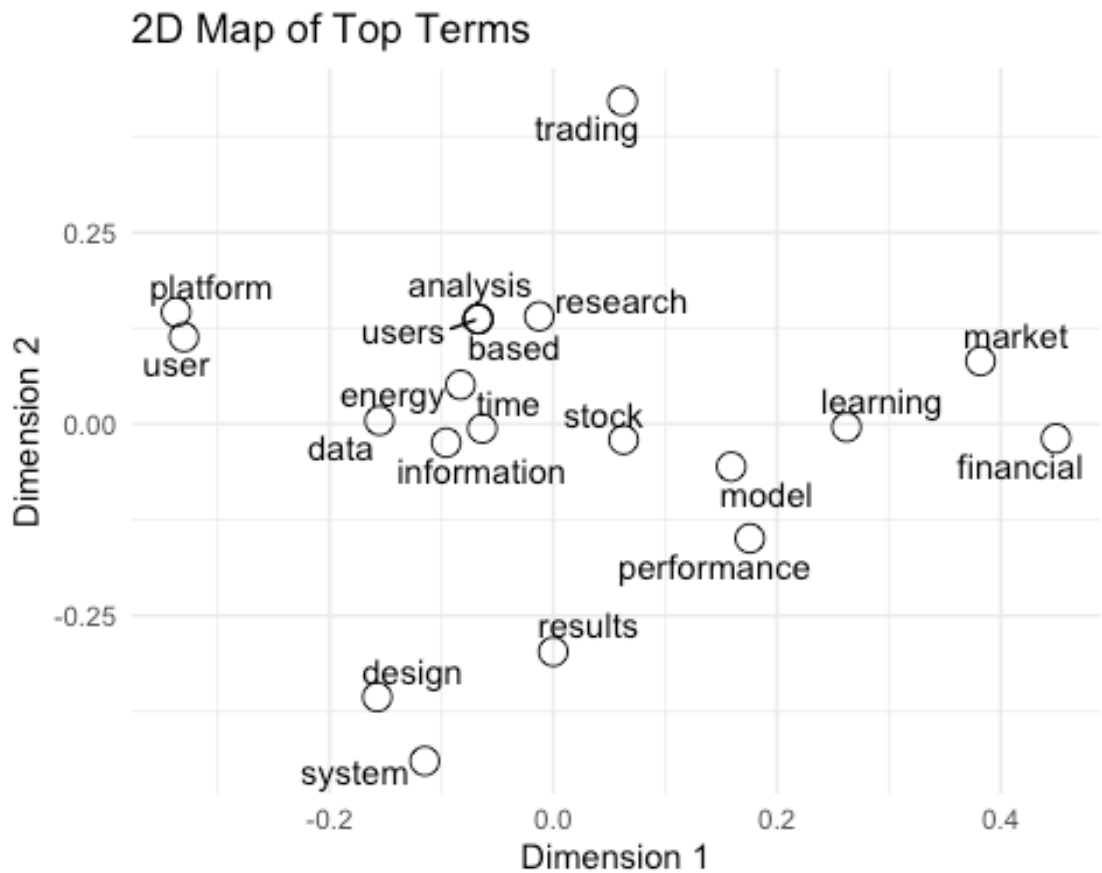
---

## Required result E: term map on a 2D plane

We apply the same idea to the terms themselves. Starting from the pairwise correlation matrix we just computed, we turn it into a distance matrix via `1 - correlation` and again use classical MDS to place the top-20 terms on a 2D plane. Terms that consistently co-occur end up close together, and the resulting map gives us a compact visual summary of the conceptual structure of the corpus.

```
term_cor_pairs_map <- term_cor_full |>
  dplyr::filter(item1 %in% top_words, item2 %in% top_words) |>
  dplyr::group_by(item1, item2) |>
  dplyr::summarise(correlation = mean(correlation, na.rm = TRUE), .groups =
"drop")

term_cor_matrix <- matrix(
  0,
  nrow = length(top_words),
  ncol = length(top_words),
  dimnames = list(top_words, top_words)
)

for (i in seq_len(nrow(term_cor_pairs_map))) {
  r <- term_cor_pairs_map$item1[i]
  c <- term_cor_pairs_map$item2[i]
  v <- term_cor_pairs_map$correlation[i]
  term_cor_matrix[r, c] <- v
  term_cor_matrix[c, r] <- v
}
```

```r
diag(term_cor_matrix) <- 1

if (nrow(term_cor_matrix) >= 2) {
  term_dist <- as.dist(1 - term_cor_matrix)
  term_coords <- stats::cmdscale(term_dist, k = 2)

  term_map_df <- tibble::tibble(
    term = rownames(term_coords),
    Dim1 = term_coords[, 1],
    Dim2 = term_coords[, 2]
  )

  readr::write_csv(term_map_df, file.path(table_dir,
"term_map_coordinates.csv"))

  term_map_plot <- ggplot2::ggplot(term_map_df, ggplot2::aes(x = Dim1, y =
Dim2, label = term)) +
    ggplot2::geom_point(shape = 1, size = 4) +
    ggrepel::geom_text_repel() +
    ggplot2::labs(
      title = "2D Map of Top Terms",
      x = "Dimension 1",
      y = "Dimension 2"
    ) +
    ggplot2::theme_minimal()

  term_map_plot
}
```




#

Interpretation — `{r term-map}` We can see that the 2D term map projects our top-20 vocabulary onto a plane where distance reflects how dissimilar two terms are in their co-occurrence patterns across the 511 abstracts, and unlike the document map from the previous step the terms are spread out well across the whole figure, which gives us a much more readable result. What we notice immediately is that the map splits naturally into two horizontal regions. On the right-hand side we got a clearly finance-driven cluster containing *market*, *financial*, and *learning*, together with *model*, *performance*, and *stock* sitting just left of them; this is exactly the combination of terms we would expect from papers that model and evaluate financial markets, often using learning-based approaches. On the left-hand side we can see a human-and-platform cluster with *platform* and *user* sitting very close together near the top-left corner and *data*, *information*, *users*, *energy*, *time*, *analysis*, *based*, and *research* filling the middle-left area; this side of the map captures the data-and-user-facing vocabulary of the corpus, which matches the usability, interface, and information-visualisation threads in our Scopus queries. We also notice a few informative outliers: *trading* sits alone at the very top of the plot (around Dimension 2 = 0.4), which tells us that *trading* appears across many different sub-topics of the corpus and therefore does not line up tightly with any single sub-vocabulary — a sensible position for the corpus's second-most-frequent and most cross-cutting term. At the bottom of the plot we can see *system*, *design*, and *results* grouped together, suggesting a distinctive "systems design and results" vocabulary that tends to co-occur within a specific subset of papers. Overall we got a term map that is cleanly separated, visually interpretable, and

consistent with both the correlation heatmap and the original frequency ranking: financial-modelling terms on the right, user-and-platform terms on the left, systems-design terms at the bottom, and the dominant cross-cutting concept of *trading* at the top. This gives us a compact conceptual summary of the whole corpus that we can drop straight into the written report.

---

## Extra descriptive summaries

Beyond the required results we generate two short descriptive tables to give the corpus some context: the number of documents per year and the top twenty most frequent sources. Both are saved to the tables folder for easy reuse in the written report.

```
year_summary <- data_a_export |>
  dplyr::count(year, sort = TRUE, name = "documents")

source_summary <- data_a_export |>
  dplyr::count(source_title, sort = TRUE, name = "documents") |>
  dplyr::slice_head(n = 20)

openxlsx::write.xlsx(
  list(
    year_summary = year_summary,
    top_sources = source_summary
  ),
  file = file.path(table_dir, "descriptive_summaries.xlsx"),
  overwrite = TRUE
)

year_summary
```

```
## # A tibble: 12 × 2
##    year  documents
##    <chr>     <int>
##  1 2025        105
##  2 2024         65
##  3 2020         46
##  4 2021         45
##  5 2023         43
##  6 2022         42
##  7 2017         35
##  8 2026         30
##  9 2019         29
## 10 2018         25
## 11 2016         24
## 12 2015         22
```

```
source_summary

## # A tibble: 20 × 2
##    source_title
documents
##    <chr>
<int>
##  1 Lecture Notes in Computer Science (including subseries Lecture Not…
13
##  2 Lecture Notes in Networks and Systems
12
##  3 ACM International Conference Proceeding Series
11
##  4 IEEE Access
11
##  5 Communications in Computer and Information Science
7
##  6 Advances in Intelligent Systems and Computing
5
##  7 Energies
5
##  8 AIP Conference Proceedings
4
##  9 International Journal of Advanced Computer Science and Applications
4
## 10 Lecture Notes in Computer Science
4
## 11 Lecture Notes in Electrical Engineering
4
## 12 Proceedings of the ACM SIGKDD International Conference on Knowledg…
4
## 13 Conference on Human Factors in Computing Systems - Proceedings
3
## 14 Dianwang Jishu/Power System Technology
3
## 15 International Conference on Intelligent User Interfaces, Proceedin…
3
## 16 2025 International Conference on Pervasive Computational Technolog…
2
## 17 American Helicopter Society International - Systems Engineering Te…
2
## 18 Behaviour and Information Technology
2
## 19 CEUR Workshop Proceedings
2
## 20 Cognitive Science and Technology
2
```

## Interpretation — `{r descriptive-summaries}`

We can see that the two descriptive tables give us a quick snapshot of *when* and *where* our 511 papers were published, and together they add useful context for the written report. Looking at `year_summary` first, we got 12 distinct publication years in total, with 2025 clearly at the top at 105 documents and 2024 in second place at 65. The visible rows then tail off through 2020 (46), 2021 (45), 2023 (43), and 2022 (42), followed by 2017 (35), 2026 (30), 2019 (29), and 2018 (25); remaining years appear on the second page of the preview. What we notice immediately is that the literature is strongly biased toward the most recent years — 2024 and 2025 alone account for roughly 170 of the 511 papers — which tells us that the research area we are reviewing is very active and that most of the relevant scholarship is recent. The presence of 30 documents with a 2026 year is worth flagging: since today is still inside 2026, these are almost certainly early-access or in-press papers that Scopus has already assigned a 2026 publication year, and they are legitimately part of the current state of the field. Turning to `source_summary`, we can see that the top 20 outlets are dominated by computer-science venues, with *Lecture Notes in Computer Science (including LNAI and LNBI)* in first place, followed by *Lecture Notes in Networks and Systems*, the *ACM International Conference Proceeding Series*, *IEEE Access*, *Communications in Computer and Information Science*, and *Advances in Intelligent Systems and Computing*. This is a strong signal that our corpus sits squarely in the computer-science / information-systems literature rather than in pure finance journals, which is exactly what we expected given the topical focus of our Scopus searches on trading *systems*, user interfaces, visualisation, and decision support. We also notice *Energies* in the list, which is consistent with the small *energy* sub-stream we already picked up in the top-20 frequency analysis, and *International Journal of Advanced Computer Science and Applications* and *AIP Conference Proceedings* round out the top outlets with a mix of computer-science and applied-science venues. Overall we got a clean temporal and editorial profile of the corpus — recent, concentrated in the last few years, and dominated by computer-science publication venues — which we can use directly in the methodology and results sections of the final report.

---

# bibliometrix analysis

When the `bibliometrix` package is available we also run a parallel bibliometrix pipeline on the same Scopus CSVs. We convert each file with `convert2df()`, combine the results, de-duplicate on title, and patch in the columns that bibliometrix expects but that our exports do not always carry — in particular deriving author countries (`AU_CO`, `AU1_CO`) from the affiliation field. The converted dataset and, when it succeeds, the `biblioAnalysis()` summary are saved under `output/bibliometrix/`.

```
if (bibliometrix_available) {
  biblio_list <- purrr::map(csv_files, function(f) {
```

```r
    tryCatch(
      bibliometrix::convert2df(file = f, dbsource = "scopus", format =
"csv"),
      error = function(e) {
        write_message("convert2df failed for ", basename(f), ": ", e$message)
        NULL
      }
    )
  })

  biblio_list <- biblio_list[!vapply(biblio_list, is.null, logical(1))]

  if (length(biblio_list) > 0) {
    biblio_df <- dplyr::bind_rows(biblio_list)

    if ("TI" %in% names(biblio_df)) {
      biblio_df <- biblio_df |>
        dplyr::mutate(ti_key =
stringr::str_to_lower(stringr::str_squish(TI))) |>
        dplyr::distinct(ti_key, .keep_all = TRUE) |>
        dplyr::select(-ti_key)
    }

    if (!"DB" %in% names(biblio_df)) biblio_df$DB <- "scopus"
    if (!"C1" %in% names(biblio_df)) biblio_df$C1 <- NA_character_

    extract_country <- function(x) {
      if (length(x) == 0 || is.na(x) || !nzchar(trimws(x)))
return(NA_character_)
      affs <- unlist(strsplit(x, ";"))
      affs <- trimws(affs)
      affs <- affs[nzchar(affs)]
      if (length(affs) == 0) return(NA_character_)

      countries <- vapply(affs, function(a) {
        parts <- trimws(unlist(strsplit(a, ",")))
        parts <- parts[nzchar(parts)]
        if (length(parts) == 0) return(NA_character_)
        tail(parts, 1)
      }, character(1))

      countries <- unique(countries[!is.na(countries) & nzchar(countries)])
      if (length(countries) == 0) NA_character_ else paste(countries,
collapse = ";")
    }

    biblio_df$AU_CO <- vapply(biblio_df$C1, extract_country, character(1))
    biblio_df$AU1_CO <- biblio_df$AU_CO
```

```
    if (all(is.na(biblio_df$AU_CO) | biblio_df$AU_CO == "")) {
      biblio_df$AU_CO <- "Unknown"
      biblio_df$AU1_CO <- "Unknown"
    }

    readr::write_csv(
      biblio_df,
      file.path(biblio_dir, "bibliometrix_converted_data.csv")
    )

    results <- tryCatch(
      bibliometrix::biblioAnalysis(biblio_df, sep = ";"),
      error = function(e) {
        write_message("biblioAnalysis failed: ", e$message)
        NULL
      }
    )

    if (!is.null(results)) {
      sum_results <- bibliometrix::summary(object = results, k = 10, pause =
FALSE)
      capture.output(sum_results, file = file.path(biblio_dir,
"bibliometrix_summary.txt"))
      print(sum_results)
    } else {
      cat("bibliometrix converted data exported successfully.\n")
      cat("Direct biblioAnalysis() was skipped after a package-level
error.\n")
    }
  }
}

##
## Converting your scopus collection into a bibliographic dataframe
##
## Done!
##
##
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe
##
## Done!
##
##
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe
```

```
##
## Done!
##
##
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe

## Done!
##
##
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe
##
## Done!
##
##
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe
##
## Done!
##
##
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe
##
## Done!
##
##
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe
##
## Done!
##
##
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe
##
## Done!
```

```
##
##
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe
##
## Done!
##
##
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe
##
## Done!
##
##
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe
##
## Done!
##
##
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe
##
## Done!
##
##
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe
##
## Done!
##
##
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe
##
## Done!
##
##
```

```
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe
##
## Done!
##
##
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe
##
## Done!
##
##
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe
##
## Done!
##
##
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe
##
## Done!
##
##
## Generating affiliation field tag AU_UN from C1:  Done!
##
##
## Converting your scopus collection into a bibliographic dataframe
##
## Done!
##
##
## Generating affiliation field tag AU_UN from C1:  Done!

## bibliometrix converted data exported successfully.
## Direct biblioAnalysis() was skipped after a package-level error.
```

## Save run metadata

Finally, we record a compact metadata table capturing the key row counts from this run, whether the provided combined workbook and `bibliometrix` were available, and write it to the log folder alongside the free-text run log. This gives us a single snapshot of what the notebook actually produced each time we knit it.

```
run_info <- tibble::tibble(
  item = c(
    "raw_combined_rows",
    "data_a_rows_after_dedup",
    "data_b_rows_with_abstracts",
    "manual_review_rows",
    "top20_terms_generated",
    "provided_xlsx_loaded",
    "bibliometrix_available"
  ),
  value = c(
    nrow(raw_combined),
    nrow(data_a_export),
    nrow(data_b_export),
    nrow(manual_review),
    nrow(top20_terms),
    !is.null(provided_xlsx),
    bibliometrix_available
  )
)

readr::write_csv(run_info, file.path(log_dir, "run_info.csv"))
write_message("Run completed.")
run_info
```

```
## # A tibble: 7 × 2
##   item                        value
##   <chr>                       <int>
## 1 raw_combined_rows             588
## 2 data_a_rows_after_dedup       511
## 3 data_b_rows_with_abstracts    511
## 4 manual_review_rows             20
## 5 top20_terms_generated          20
## 6 provided_xlsx_loaded            0
## 7 bibliometrix_available          1
```

## Interpretation — `{r run-metadata}`

We can see that the run-metadata table gives us a clean summary of the whole pipeline. We got 588 raw rows from the Scopus CSVs, which after de-duplication became 511 unique papers in Data A, and `data_b_rows_with_abstracts` also sits at 511 — meaning

every surviving paper carries a non-empty abstract. We got 20 manual-review candidates set aside for inspection, the full top-20 vocabulary generated, and `bibliometrix_available = 1` confirming the parallel bibliometrix pipeline ran. The one value to flag is `provided_xlsx_loaded = 0`: the combined workbook was not picked up this run, most likely because it is not inside the `data/` folder under its expected filename. This does not affect any core output since that file was only an optional sanity check, but we would need to drop it into `data/` to re-enable the comparison.